

RUNTIME ARCHITECTURE MAP

Every part and sub-part of the Sofuu runtime — from the **C core** (QuickJS, libuv, the QTSQ codec) through the **Rust shell** and its **five on-device ML models**, up to the **shipped JavaScript drivers** and the **macOS desktop app**, mapped as a visual hierarchy and a set of reference tables.

素風

The Big Picture

Sofuu is a server-side JavaScript runtime built in Rust around a C core. Everything an AI-era app normally installs from npm — LLM streaming, agents, MCP, web search, HTTP — is built in below the language surface.

2.5MB

RELEASE BINARY

~3ms

COLD STARTUP

53.2k

LINES OF RUST

5.0k

LINES OF SHIPPED JS

THE LAYER STACK – TOP-LEVEL HIERARCHY



Every Part, One Map



☐ Rust-owned part ☐ C or C-boundary part

rt/ (20) = 20 source files in the folder

C Core & the FFI Bridge

Layer 0 and Layer 1. The C core is vendored under `deps/` plus the QTSQ store codec; `sofuu-ffi` wraps every C symbol in safe Rust; `sofuu-capi` turns the whole runtime into an embeddable C library.

PART	SUB-PART	ROLE IN THE RUNTIME
DEPS/ – VENDED C DEPENDENCIES (LAYER 0)		
deps/quickjs	JS engine	The JavaScript engine itself — parses and executes all user code, both at the CLI and inside RLM sandboxes.
deps/libuv	event loop	Async I/O: timers, file I/O, sockets, subprocesses. The Rust <code>rt/loop.rs</code> interleaves its loop with QuickJS microtasks.
deps/llhttp	HTTP parsing	Request/response parsing for the HTTP server and client wire paths.
deps/http-parser	HTTP parsing (vendored)	Legacy parser kept for the server path; bound in <code>sofuu-ffi/src/http_parser.rs</code> .
QTSQ codec	binary store format	Lives in its own repo (black-hole-disk); compressed, checksummed containers for the <code>.qtsq</code> stores — brain, sessions, audits.
CRATES/SOFUU-FFI – SAFE RUST BINDINGS TO THE C CORE (LAYER 1)		
src/lib.rs	facade	The safe-Rust facade over the whole C core — every other crate calls through here.
src/build.rs	build script	Compiles the C core (QuickJS + <code>src/*.c</code> + <code>libuv</code> + <code>curl</code>) into static libraries; guards against QTSQ layout drift.
src/qjs.rs	QuickJS bindings	Engine bindings — the M0 keystone of the Rust migration.
src/qjs_rt.rs	sandbox runtime	Hermetic QuickJS sandbox runtime used by the RLM loop (nested, isolated contexts).
src/uv.rs	libuv bindings	Event-loop and handle bindings backing all async Rust-side I/O.
src/curl.rs	libcurl bindings	HTTP client transport — including the UAF-hardened streaming callbacks behind <code>fetch</code> and AI streams.
src/http_parser.rs	parser bindings	Bindings to the vendored <code>http-parser</code> used by the HTTP server.
src/qtsq.rs	codec bindings	Read/write of <code>.qtsq</code> containers — the project store format for brain and sessions.
src/bridge.rs	engine bridge	QuickJS bridge bindings connecting engine callbacks into Rust.
CRATES/SOFUU-CAPI – THE EMBEDDABLE C ABI (LAYER 1)		
src/lib.rs	C ABI surface	Embeddable C ABI (PLAN-HEADLESS H1) — host applications can run the Sofuu engine as a library.

The Runtime Substrate — `sofuu-core/src/rt/`

Twenty files that make the engine tick: the event loop, every I/O primitive exposed to JS, the AI and MCP modules, and the capability registries that keep requests inside each model’s real limits.

SUB-PART (FILE)	ROLE IN THE RUNTIME
<code>mod.rs</code>	Module root — the Rust event-loop substrate (libuv loop + microtask interleaving).
<code>engine.rs</code>	QuickJS engine lifecycle + file evaluation; registers all Rust modules into JS.
<code>loop.rs</code>	Global libuv event loop with QuickJS microtask interleaving after every poll.
<code>promise.rs</code>	Promise ↔ libuv async bridge + deferred unhandled-rejection tracker.
<code>timer.rs</code>	<code>setTimeout</code> / <code>setInterval</code> / <code>clearTimeout</code> / <code>sofuu.s.sleep</code> .
<code>fs.rs</code>	Async file I/O on libuv — <code>sofuu.fs.read/write/stat</code> family.
<code>process_spawn.rs</code>	<code>sofuu.spawn</code> / <code>sofuu.exec</code> subprocesses.
<code>http_client.rs</code>	The <code>fetch</code> API over libcurl — UAF-hardened streaming response buffers.
<code>http_server.rs</code>	The <code>sofuu.http</code> HTTP/1.1 server (<code>createServer</code> , routing, streaming responses).
<code>http_sse.rs</code>	JS-facing <code>sofuu.SSEParser</code> shell over the Rust SSE parser.
<code>ai.rs</code>	The <code>sofuu.ai</code> module — streaming LLM calls across OpenAI / Anthropic / ollama wires.
<code>mcp.rs</code>	MCP module — client + server, tools/resources/prompts over JSON-RPC.
<code>memory.rs</code>	Memory subsystem JS module — CMA cognitive memory, key-value store, agent memory.
<code>model_caps.rs</code>	Per-model capability registry — context window, max output, thinking kind; single source of truth.
<code>model_caps_discovered.rs</code>	Caps the endpoint itself publishes — a discovery ladder between learned-400 limits and the registry.
<code>cjs.rs</code>	CommonJS compatibility shim (<code>require/module.exports</code>) for legacy packages.
<code>npm.rs</code>	npm package resolution + installer (port of the C resolver, semantics verbatim).
<code>session_js.rs</code>	Synchronous session-mesh primitives exposed to shipped JS (cross-session context sharing).
<code>host_poke.rs</code>	Thread-safe host→engine wakeup — how the desktop app injects turns into the blocking engine thread.
<code>tui.rs</code>	Minimal alt-screen TUI for interactive chat (port of the retired C TUI, same render model).

Core Subsystems & Top-Level Files

The rest of `sofuu-core/src/`: protocol subsystems, the cognitive memory stack, the RLM scaffolding, JS-visible module shells, and the single-file heavyweights that tie it together.

PART	SUB-PART	ROLE IN THE RUNTIME
HTTP/ — HTTP SUBSYSTEM		
<code>http/mod.rs</code>	subsystem root	Shared HTTP types and setup for client + server paths.
<code>http/sse.rs</code>	SSE parser	Streaming Server-Sent Events parser (Rust port of the C parser) — feeds AI streams and MCP.
MCP/ — MODEL CONTEXT PROTOCOL		
<code>mcp/mod.rs</code>	subsystem root	MCP client + server lifecycle over JSON-RPC 2.0.
<code>mcp/jsonrpc.rs</code>	JSON-RPC layer	Security-sensitive request/notification/response parsing (fuzz target).
MEMORY/ — COGNITIVE MEMORY STACK		
<code>memory/cma.rs</code>	CMA	Cognitive Memory Architecture — the 4-tier memory model (working / episodic / semantic / archive).
<code>memory/hnsw.rs</code>	HNSW index	Hierarchical Navigable Small World vector index powering CMA recall.
RLM/ — RECURSIVE LANGUAGE MODEL LOOP		
<code>rlm/sandbox.rs</code>	sandbox	Hermetic nested QuickJS sandbox — the context lives as data inside it.
<code>rlm/episode.rs</code>	episode loop	The RLM state machine; the model stays external (network), never inside the loop.
<code>rlm/router.rs</code>	router	Routes a turn between a plain model call and the RLM loop; heuristics first, logged for future learning.
<code>rlm/js_api.rs</code>	JS API	The engine seam that registers RLM + shipped JS drivers into QuickJS.
MODULES/ — JS-VISIBLE MODULE SHELLS		
<code>modules/console.rs</code>	console	<code>console.log/info/warn/error/assert/time/table</code> .
<code>modules/process.rs</code>	process	process object, TTY readline, and the chat bridges into the TUI driver.
TOP-LEVEL SINGLE FILES		
<code>chat.rs</code>	chat driver	The Rust interactive chat driver — turn engine, TUI phases, provider failover, resume.
<code>session.rs</code>	session mesh	Multi-session real-time context sharing, scoped to one project (git root).
<code>bundler.rs</code>	ESM bundler	Resolves an ESM import graph into a single executable bundle.
<code>ts.rs</code>	TS stripper	TypeScript type stripper — replaces type-only syntax with spaces (fuzz target).
<code>npm.rs</code>	npm safe core	Security-sensitive pure logic of npm resolution (spec parsing — fuzz target).
<code>shipped.rs</code>	driver registrar	Registers the shipped runtime JS (agent, chat, tools, web) into the engine via <code>include_str!</code> .
<code>embed_config.rs</code>	embed config	Process-global config for embedded mode when Sofuu runs as a library.
<code>ffi_exports.rs</code>	C-ABI exports	C-ABI exports letting the remaining C core call into safe Rust mid-migration.

The ML Suite — Five Tiny On-Device Models

Each gate is a separate ~8.5k-parameter MLP trained offline by `ml-train`, baked into the binary, and strictly advise-only: `/ml off` (or `SOFUU_NO_ML=1`) restores the mechanical ratios exactly.

MODEL 1 ml/freshness Is this material current? Scores staleness of recalled context before it enters the prompt.	GATE ml/relevance Pre-retrieval relevance gate — decides what is worth fetching into context at all.
GATE ml/supervisor Real-time supervisor over tool calls — flags likely-wasted calls as they happen, with guarded online learning.	MODEL 4 ml/compaction When and what to compact — replaces the old fixed 70% cliff with a learned trigger.
MODEL 5 · NEWEST ml/alloc Context-window allocator — fetches the selected model's real caps first (registry → learned limits → defaults), then allocates per turn: compact point, tool-result cap, recall budget, attach budget, output reserve. Paired with a mechanical Layer-0 clamp and a Layer-1 pre-flight fit ladder so an oversized request never reaches the provider.	

SUB-PART (FILE)	ROLE IN THE SUITE
<code>ml/net.rs</code>	The generic tiny MLP shared by every gate (24→96→40→1 shape family).
<code>ml/online.rs</code>	Online adaptation of the supervisor's output layer — confidence-weighted labels, gold = 2.0, two-step adopt/discard, 31-fixture gate.
<code>ml/context.rs</code>	Shared in-memory context working set — structured view of what the agent loop currently holds.
<code>ml/*/features.rs</code>	One feature extractor per family — 24+ numeric features from caps, history state, and task shape.
<code>ml/*/model.rs</code>	Per-family model wrapper — loads baked weights (include_bytes!), runs inference.
<code>ml/*/eval.rs</code>	Per-family evaluation harness — fixture gates + property tests (no plan exceeds hard limits).

crates/ml-train

The offline trainer: `train.rs` plus one synthetic-data generator per family (`data_alloc` / `data_compaction` / `data_freshness` / `data_relevance` / `data_supervisor`) — 4,810+ labeled trajectories, channel-coverage splits so val/test families never leak into train.

Shipped JavaScript & the Desktop App

Layer 3 embeds five JS drivers straight into the binary; Layer 4’s macOS app wraps the same engine in Tauri with a permission gate and a dedicated engine thread.

SRC/JS/ — SHIPPED DRIVERS (EMBEDDED VIA INCLUDE_STR!)

SUB-PART	LINES	ROLE IN THE RUNTIME
agent.js	2,160	Agent runtime — agent definitions, the tool loop, sub-agents, traces (sofuu.agent).
chat.js	1,787	Chat turn engine — streaming, alloc pre-flight ladder, provider failover, session resume.
tools.js	428	Built-in coding tools — read / write / edit / grep / glob / list_dir / bash.
web.js	327	sofuu.web — web_search + web_open; keyless DuckDuckGo default.
rlm.js	305	RLM protocol running inside the hermetic sandbox (tool()) dispatch, replies).

SOFUU-DESKTOP/SRC-TAURI/ — TAURI BACKEND (RUST)

SUB-PART	ROLE
engine.rs	Dedicated engine worker thread — the UI never blocks on the runtime.
commands.rs	The Tauri command surface — every IPC entry point the WebView can call.
host.rs	The JS→desktop event sink — engine events stream out to the UI.
keychain.rs	Provider API keys in the macOS Keychain — keys never reach the WebView.
config.rs / sessions.rs	Desktop config access; read-only mirror of the session registry.

SOFUU-DESKTOP/SRC/ — REACT UI (13 COMPONENTS + LIB)

SUB-PART	ROLE
ChatPane · Composer · TopBar · Sidebar	Core shell — message rows, input composer, session pills, workspace sidebar (liquid-glass).
ToolTimeline · ApprovalDialog · DiffView	Agent activity tree, the allow/deny permission gate, and code diffs for edits.
ContextRing · SettingsModal · Popover	Context-pressure ring, provider/model settings, drill-in pickers.
CloudBackground · Icon · ErrorBoundary	WebGL procedural clouds (never loops), icon set, crash containment.
lib/ (backend · events · store · errlog · shortcuts)	IPC bridge to Tauri, event typing, UI state store, error log, keybindings.

Support Crates, Storage & Periphery

SUPPORT CRATES + THE PROJECT STORE

PART	SUB-PARTS	ROLE
crates/ml-train	train.rs + 5 data_*.rs	Offline trainer — synthetic trajectory generators, split discipline, weight baking for all five models.
crates/fuzz	7 fuzz targets	cargo-fuzz over the trust boundaries: capi_call, jsonrpc, npm_spec, rlm_reply, rlm_snippet, sse, ts_strip.
<project>/sofuu/	brain · sessions · audits · debug · issues	QTSQ-only project store — per-session conversation files written in real time, plus debug/issue/audit records.

HOW THE PIECES CONNECT — THE THREE SEAMS

SEAM	WHAT CROSSES IT
C ↔ Rust	sofuu-ffi: QuickJS eval + handles, libuv loop polling, libcurl transfers, QTSQ container read/write.
Rust ↔ JS	rt/engine.rs registers every sofuu.* module; shipped.rs embeds the five JS drivers; host_poke wakes the engine thread.
Engine ↔ UI	Desktop only: commands.rs in, host.rs events out — the WebView never sees API keys or raw engine state.

PERIPHERY — IN THE REPO, OUTSIDE THE RUNTIME

FOLDER	CONTENTS
examples/	~26 runnable demos — HTTP servers, AI streaming, agents, MCP, SIMD.
tests/	Agent test suite (171+ checks), chat dual-path E2E, resume/failover drivers, live-provider probes.
bench/ + benchmarks/	Startup, throughput, and SIMD micro-benchmarks.
scripts/ + tools/	Build/dev helper scripts and dev tooling.
sofuu-landing/	The marketing site (separate deployable).

reading order

New to the codebase: start at Layer 0 on the map (page 3), then follow the tables upward — C core & FFI (p.4), runtime substrate (p.5), subsystems (p.6), ML suite (p.7), JS & desktop (p.8).